DSA03-NATURAL LANGUAGE PROCESSING

ASSIGNMENT-3

ASSIGNMENT 3 – CO3

CO3 Coverage: Context-Free Grammars for English syntax; Context-free rules and trees; Sentence-level constructions; Agreement; Subcategorization; Parsing (Top-down, Earley); Feature structures; Probabilistic Context-Free Grammars

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Assignment Title: Design, Parse and Probabilistically Rank English Queries for a University Helpdesk Using Context-Free Grammars, Feature Structures and Earley Parsing (Total Marks – 50)

Duration: 1 week

Scenario — Syntactic Analysis for a University Helpdesk Assistant

A university is building a text-based helpdesk assistant that answers student and staff queries about timetables, examinations, hostels, fees, library services and placements. Before any meaning can be extracted, every query must be assigned a reliable syntactic structure. The NLP team has decided to build a grammar-based parser for a controlled but realistic fragment of English, to resolve ambiguity with a probabilistic model, and to compare parsing algorithms on accuracy and cost.

You are the language-engineering team. You must investigate how English syntax can be captured by context-free rules, how agreement and subcategorization constrain those rules, how parsing algorithms search for trees, and how probabilities select the best tree when a query is ambiguous.

Planning data and constraints

- Grammar fragment: at least 40 phrase-structure rules and a lexicon of at least 60 words covering the helpdesk domain.
- Sample query set (build and document your own): at least 30 sentences, including declaratives, imperatives, yes/no questions, wh-questions and at

least 5 structurally ambiguous sentences (e.g., “Show the students with the late fee notice”, “Book the room near the library for the seminar”).

- Required phenomena: subject–verb agreement, determiner–noun number agreement, pronoun case, and at least five verb subcategorization frames.
- The grammar must be written in Chomsky-style CFG notation; for the Earley and probabilistic tasks the grammar may be converted to a binarised or CNF-equivalent form where needed, and the conversion must be shown.
- Treebank probabilities: a small hand-annotated corpus of at least 20 parsed sentences may be used to estimate rule probabilities (maximum-likelihood estimates with smoothing explained).
- Any manual effort (grammar writing, annotation, testing) must be accounted for in your effort and cost discussion.

Design Challenge (Day 4):

The helpdesk is to be extended to accept noisy, conversational and multi-clause student messages (e.g., “Can you tell me when the exams that were postponed will be held?”). Re-evaluate your original grammar and parser and decide whether to retain, modify or redesign them.

TASK 1 – Design a Context-Free Grammar for English Syntax 5 Marks Guiding

Enquiry

How can the core structure of English be captured by a finite set of context-free rules, and what design decisions make the grammar both expressive and manageable?

Investigate and interpret:

- Define the formal CFG **G = (N, Σ, R, S)** for your helpdesk fragment and state clearly the non-terminals, terminals (lexicon), rules and start symbol.
- Write rules for the major constituents: sentence (S), noun phrase (NP), verb phrase (VP), prepositional phrase (PP), adjective phrase, and coordination. Include recursive rules (e.g., NP → NP PP) and explain how they permit unbounded structure.
- Separate the grammar proper from the lexicon; list the parts of speech used and justify the granularity you selected.
- Explain the notions of constituency, head, and phrasal category with tests (substitution, movement, coordination) applied to at least three of your sentences.
- Show that your grammar is context-free by construction and discuss one English phenomenon that strains the context-free assumption.

Evidence Expected

A complete, well-organised CFG with lexicon, justified categories and worked constituency tests.

TASK 2 – Context-Free Rules, Derivations and Parse Trees 3 Marks Guiding

Enquiry

How are sentences generated from the grammar, and how are trees used to represent and compare syntactic analyses?

Investigate and interpret:

- For five sentences from your query set, give a leftmost derivation and a rightmost derivation, and draw the corresponding parse tree.
- Represent each tree in both bracketed notation and tree diagram form, and compute its height, number of nodes and number of rule applications.
- Demonstrate ambiguity: for at least two sentences, give two distinct parse trees and describe the difference (e.g., PP-attachment, coordination scope).
- Explain the relationship between derivations and trees, and why many derivations may correspond to one tree.
- State a procedure to test whether a given string belongs to L(G) and discuss its limits.

Evidence Expected

Correct derivations, accurately drawn trees, and a clear analysis of structural ambiguity.

TASK 3 – Sentence-Level Constructions 4 Marks Guiding

Enquiry

Which sentence types must the helpdesk understand, and how does the grammar represent each of them?

Investigate and interpret:

- Provide rules for the four main sentence-level constructions: declaratives, imperatives, yes/no questions and wh-questions (subject and non-subject).
- Handle auxiliary inversion, wh-movement (represented through a gap or an additional rule), and subordinate and relative clauses (e.g., “the exams that were postponed”).
- Show parse trees for at least one sentence of each construction type and note the structural differences from the declarative.
- Discuss how coordination (“fees and hostel charges”) and long-distance dependencies are treated and where the plain CFG becomes unwieldy.
- Analyse the effect of each new construction on grammar size and ambiguity.

Evidence Expected

Rules and trees for each sentence type, with an analysis of how the added constructions affect the grammar.

TASK 4 – Agreement 3 Marks Guiding

Enquiry

How can a grammar prevent sentences such as "*The students submits the form" without producing an enormous set of rules?

Investigate and interpret:

- Identify the agreement phenomena in your fragment: subject–verb (number, person), determiner–noun, and pronoun case.
- First, enforce agreement by multiplying non-terminals (e.g., S → NP-sg VP-sg | NP-pl VP-pl). Count how many rules this requires for your grammar and explain the growth.
- Show how the rule explosion worsens as person, number, case and gender are combined, and compute the number of rules needed in the worst case.
- Propose a more economical alternative (feature-based rules) and show informally how it prevents ungrammatical strings; the formal treatment follows in Task 8.
- Test the original and agreement-aware grammars on at least six grammatical and six ungrammatical sentences and tabulate the results.

Evidence Expected

A quantified comparison of the baseline and agreement-aware grammars with test results.

TASK 5 – Subcategorization 4 Marks

Guiding Enquiry

How does the grammar know which complements a verb may take, and how can this knowledge be encoded?

Investigate and interpret:

- Define subcategorization (valence) and list at least five frames for helpdesk verbs, e.g., intransitive ("The portal crashed"), transitive ("Submit the form"), ditransitive ("Send me the receipt"), sentential complement ("Tell me that the fee is paid"), and PP complement ("Apply for a hostel room").
- Encode the frames first with distinct verb categories (V-intr, V-trans, V-ditrans, V-comp, …) and then explain the drawbacks of this approach.
- Distinguish complements from adjuncts using tests, with examples.

- Demonstrate how the grammar rejects over- and under-complemented sentences ("*Submit", "*The portal crashed the form") and show this in the parse results.
- Discuss verbs with multiple frames and how this changes ambiguity and parsing cost.

Evidence Expected

A subcategorization lexicon, grammar rules enforcing it, and tested accept/reject examples.

TASK 6 – Top-Down Parsing 4 Marks Guiding

Enquiry

How does a top-down parser search for a tree rooted in S, and what are the strengths and weaknesses of this strategy?

Investigate and interpret:

- Describe the top-down (recursive-descent, goal-driven) algorithm in pseudocode, including how it expands non-terminals, matches words and backtracks.
- Trace the parser step by step on one unambiguous and one ambiguous sentence, showing the stack/agenda, the rule chosen and each backtracking point.
- Explain the problems of left recursion (e.g., NP → NP PP) and repeated re-parsing of the same subtrees, and show these problems on a concrete example.
- Show how left-recursion elimination or a bottom-up filter (left-corner) addresses these issues and discuss the effect on the resulting trees.
- Analyse the worst-case time behaviour and the number of redundant sub-parses for a sentence with k prepositional phrases.

Evidence Expected

Pseudocode, detailed traces and a correct analysis of the weaknesses of top-down parsing.

TASK 7 – Earley Parsing 5 Marks

Guiding Enquiry

How does the Earley algorithm use dynamic programming to avoid the failures of simple top-down parsing, and what does its chart contain?

Investigate and interpret:

- Define an Earley state (dotted rule with start position) and explain the three operations: PREDICTOR, SCANNER and COMPLETER.

- Hand-simulate the algorithm on a helpdesk sentence of at least six words; show every chart entry (S[0]…S[n]) with its state, origin and the operation that created it.
- Recover the parse tree(s) from the chart by adding back-pointers and show how multiple trees are packed for an ambiguous sentence.
- Show how the algorithm handles a left-recursive rule and an ε-free ambiguous attachment without looping.
- Derive the complexity ($O(n^3)$ time, $O(n^2)$ space for the chart) and compare it with the top-down parser of Task 6 on the same sentence in terms of number of states generated.

Evidence Expected

A fully traced Earley chart, back-pointer tree recovery, and a justified complexity comparison.

TASK 8 – Feature Structures and Unification 4 Marks

Guiding Enquiry

How can attribute–value structures and unification replace multiplied categories to handle agreement and subcategorization elegantly?

Investigate and interpret:

- Define feature structures (attribute–value matrices), feature paths, reentrancy (structure sharing) and represent at least five lexical entries with features such as NUM, PER, CASE and SUBCAT.
- Define unification and subsumption, and work through at least three unifications, including one that fails.
- Augment at least six grammar rules with feature constraints (e.g., S → NP VP with ⟨ NP AGREEMENT⟩ = ⟨ VP AGREEMENT⟩) and show the unified feature structure for one complete parse.
- Redo Task 4 (agreement) and Task 5 (subcategorization) using features and compare the grammar size with the earlier non-feature version.
- Describe how unification can be integrated with the Earley parser (unification during the COMPLETER step) and what is lost in complexity.

Evidence Expected

Correct feature structures and unifications, a feature-based grammar fragment, and a comparison with the non-feature grammar.

TASK 9 – Probabilistic Context-Free Grammars 5 Marks

Guiding Enquiry

When a query has several valid parses, how can probabilities select the most plausible one?

Investigate and interpret:

- Define a PCFG: each rule $A \rightarrow \beta$ has a probability $P(A \rightarrow \beta \mid A)$, with the probabilities for each left-hand side summing to 1. Show that your grammar satisfies this condition.
- Estimate rule probabilities by maximum likelihood from your 20-sentence mini-treebank, giving the counts, the formula and the resulting table; explain how you handle unseen rules.
- Compute the probability of at least two parse trees for an ambiguous sentence (e.g., PP-attachment) and choose the best parse; compute the probability of the sentence as the sum over its trees.
- Describe the probabilistic CKY (Viterbi) algorithm in pseudocode after conversion to CNF, and compute the table for a short sentence.
- Discuss the independence assumptions of PCFGs (context-freeness, position invariance), the preference for shorter trees, and the lack of lexical sensitivity, illustrating each with a failure case.

Evidence Expected

A well-formed PCFG with estimated probabilities, worked tree probability calculations, a Viterbi trace and a reasoned discussion of limitations.

TASK 10 – Implement and Test the Parsers 3 Marks Guiding

Enquiry

Can a working implementation reproduce the behaviour you derived by hand?

Investigate and interpret:

- Implement in Python (plain code or NLTK) the grammar, a top-down parser, an Earley parser and a Viterbi/PCFG parser, with feature checks for agreement and subcategorization.
- Run all parsers on the 30-sentence test set and report, for each: parse found or not, number of parses, number of chart states/steps and run-time.
- Include test cases for left recursion, ambiguous attachment, ungrammatical sentences and long sentences (up to 20 words), and a reference check against your hand-computed results from Tasks 7 and 9.
- Present the results in comparable tables and at least one plot (e.g., number of states against sentence length).

Evidence Expected

Working, documented code, a reproducible test procedure and results tables/graphs.

TASK 11 – Compare, Recommend and Defend the Parsing Design 2 Marks Guiding

Enquiry

Which grammar-and-parser combination would you deploy in the helpdesk, and what evidence supports your choice?

Investigate and interpret:

- Compare the top-down, Earley and probabilistic parsers on coverage, ambiguity handling, cost and ease of grammar maintenance, using your analysis and measured results.
- Recommend a design (e.g., feature-based CFG + Earley + PCFG ranking), stating its expected accuracy and limitations.
- Discuss manual effort (grammar writing and annotation) and the point at which a statistical approach becomes more economical than hand-written rules.

Evidence Expected

A justified recommendation supported by theory and measurements; choosing the fastest option alone is not sufficient.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Changed Requirement

The helpdesk must now accept conversational, multi-clause and partly ungrammatical student messages (for example, “Hi, can you tell me when the exams that were postponed will be held, and also whether the hostel fee is due?”). Messages may contain coordinated clauses, embedded questions, relative clauses and missing punctuation. At least 20% of the incoming messages are expected to contain a construction not covered by your grammar.

Re-evaluate your original design under the changed conditions. You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

Guiding Enquiry

What exactly has changed, and which of your original assumptions no longer hold?

- Identify the constructions in the new messages that your grammar cannot parse (embedded questions, multi-clause coordination, ellipsis, fragments).

- Identify the affected assumptions: controlled vocabulary, fully grammatical input, small ambiguity, hand-written coverage.
- Estimate how the longer sentences affect the number of parses and the parsing time.

Evidence Expected

A clear analysis of how the new input exposes gaps in the original design.

B. Evaluation of the Original Design – 2 Marks

Guiding Enquiry

How would your recommended parser behave on the new input?

- Run or hand-trace at least three new messages through your original grammar and parsers and record what fails first.
- Estimate the growth of ambiguity and chart size with sentence length using your Task 7 and Task 10 results.
- Identify whether another parsing strategy handles the new input more economically.

Evidence Expected

A technically supported evaluation of the original design with traces or measurements.

C. Adaptation or Redesign – 2 Marks

Guiding Enquiry

Should the grammar and parser be extended, replaced, or supplemented?

- Choose retain, modify or redesign (e.g., add rules for embedded clauses, a robust/partial-parsing fallback, feature-based gap threading, a larger PCFG estimated from more data).
- Describe the revised grammar and parser and what changes in rules, features and probabilities.
- Determine the effect on coverage, ambiguity, parsing cost and annotation effort.

Evidence Expected

A design response with clear linguistic and algorithmic reasoning and quantified effect.

D. Evidence-Based Defence – 2 Marks

Guiding Enquiry

Why is your final decision the right one given the evidence you generated?

- Use your derivations, traces, simulation results, coverage figures and effort estimates.
- Show that the decision follows logically from the evidence and state its remaining limitations.

Evidence Expected

A well-supported final decision integrating theory, experiments and the changed requirements.

Introduction

A university helpdesk receives questions about examinations, timetables, hostels, fees, library services, placements and student accounts. Before a system can identify what a query means, it must determine how the words fit together. For example, in “Show the students with the late fee notice,” the phrase “with the late fee notice” might describe the students or explain how the helpdesk should show them. A parser represents these possible structures so a later component can select the most useful interpretation.

This assignment designs a controlled English grammar for a university helpdesk. It explains context-free grammar (CFG) rules, parse trees, sentence constructions, agreement, subcategorization, top-down parsing, Earley parsing, feature structures and probabilistic CFGs. It also considers how the system should change when students send conversational, incomplete or multi-clause messages.

The recommended initial design combines a feature-based CFG, an Earley chart parser and a probabilistic ranking model. The grammar handles common, well-formed helpdesk queries. Feature constraints check agreement and verb complements. Earley parsing shares work between possible analyses, and a PCFG ranks the resulting parses. For noisy input, the system should be extended with partial parsing and a larger, reviewed corpus. Probability calculations in this assignment are worked examples, not benchmark results from an executed parser.

Task 1 — Design a Context-Free Grammar for English Syntax

1.1 Formal definition

A context-free grammar is represented as:

G = (N, Σ, R, S)

where:

- **N** is a finite set of non-terminal symbols.
- **Σ** is a finite set of terminal symbols, such as words in the helpdesk vocabulary.
- **R** is a finite set of production rules.
- **S** is the start symbol.

For this grammar, **Q** is the start symbol because the system accepts declaratives, commands and questions. The grammar is a controlled fragment of English: a query is accepted if the rules can generate its sequence of words.

The main non-terminals are:

- **Q** — complete helpdesk query
- **S** — declarative or coordinated sentence
- **NP** — noun phrase
- **VP** — verb phrase
- **PP** — prepositional phrase
- **AP** — adjective phrase
- **IMP** — imperative

- **YNQ** — yes/no question
- **WHQ** — wh-question
- **Nbar** — noun phrase core, including noun modifiers
- **RelClause** — relative clause
- **VPbase** — verb phrase in base form after an auxiliary

The grammar separates rules from the lexicon. Rules describe how categories combine. The lexicon states which words belong to each category.

1.2 Phrase-structure rules

The following rules define the main structures used by the helpdesk. Alternatives on the right side are shorthand for separate productions.

Q → S

Q → IMP

Q → YNQ

Q → WHQ

S → NP VP

S → S CONJ S

IMP → Vbase NP

IMP → Vbase NP PP

IMP → Vbase PP

IMP → Please Vbase NP

YNQ → Aux NP VPbase

YNQ → Modal NP VPbase

WHQ → WhNP Aux VPbase

WHQ → WhAdv Aux NP VPbase

WHQ → WhNP Aux NP VPbase

WHQ → WhNP VP

NP → Det Nbar

NP → Pron

NP → Proper

NP → NP PP

NP → NP RelClause

NP → NP CONJ NP

Nbar → N

Nbar → Adj Nbar

Nbar → Nbar PP

VP → Vintr

VP → Vtrans NP

VP → Vditrans NP NP

VP → Vditrans NP PP

VP → Vpp PP

VP → Vsent S

VP → Vwh NP WHQ

VP → Aux VPbase

VP → VP PP

VP → VP AdvP

VPbase → Vbase

VPbase → Vbase NP

VPbase → Vbase NP PP

VPbase → Vbase PP

VPbase → Vbase S

VPbase → Vbase WHQ

PP → P NP

AP → Adj

AP → Adv AP

AP → AP CONJ AP

RelClause → Rel S

RelClause → Rel VP

AdvP → Adv

AdvP → Adv AdvP

VP → VP CONJ VP

These rules contain recursive productions. For example, **NP → NP PP** allows a noun phrase to take a prepositional phrase. The result can be extended again, so the grammar can generate phrases such as “the room near the library beside the examination hall.” The grammar does not need a separate rule for every possible number of modifiers.

1.3 Lexicon

The following helpdesk lexicon contains more than 60 word forms.

ategory	**xample entries**
eterminers	e, a, an, this, that, these, those, my, your, each, every, some
ouns	udent, students, staff, professor, form, forms, fee, fees, hostel, room, library, book, notice, timetable, exam, exams, result, results, course, placement, seminar, receipt, portal, deadline, schedule, campus, application, charge, query, office, card, account, scholarship, class, date, paper, hall
erbs	ıow, find, check, submit, book, reserve, tell, ask, send, give, pay, apply, register, explain, know, confirm, open, close, crash, arrive, be, hold, postpone, need, want, cost, include
ronouns	we, you, he, she, it, they, me, us, him, her, them
'h-words	ho, whom, what, which, when, where, why, how
repositions	, for, with, near, in, on, at, from, about, by, of, after, before
onjunctions	ıd, or, but
uxiliaries	ɔ, does, did, can, could, will, would, is, are, was, were, has, have, had
djectives and adverbs	te, available, current, final, new, required, postponed, online, quickly, tomorrow

A practical parser must also account for word forms such as “submits,” “submitted” and “submitting.” A morphological analyzer can map them to a base form and record grammatical information such as tense, person and number.

1.4 Constituency, heads and phrase categories

A **constituent** is a group of words that functions as a unit in a sentence. A phrase’s **head** determines its category and often its central meaning. In “the library,” the noun “library” is the head of the NP. In “near the library,” the preposition “near” is the head of the PP.

Three constituency tests demonstrate this idea:

1. **Substitution:** In “The students checked the timetable,” the NP “The students” can be replaced by “They”: “They checked the timetable.” This suggests that the words “The students” form an NP.
2. **Movement:** In “Book the seminar room near the library,” the PP “near the library” can move to the beginning: “Near the library, book the seminar room.” This supports treating

"near the library" as a constituent.

3. **Coordination:** In "fees and hostel charges," two noun phrases are joined by "and." The resulting phrase behaves like an NP in a sentence: "Fees and hostel charges are due."

The grammar is context-free by construction: every production rewrites one non-terminal without checking the symbols around it. Some language patterns, including long-distance agreement and cross-serial dependencies, are difficult to represent compactly with a plain CFG. Feature structures can express many useful constraints, but a large grammar may require considerable manual design.

Task 2 — Context-Free Rules, Derivations and Parse Trees

2.1 Derivations and trees

A **derivation** is the sequence of rule applications used to generate a sentence. A **parse tree** shows the same rule applications as a hierarchy. A leftmost derivation always expands the leftmost unfinished non-terminal. A rightmost derivation always expands the rightmost unfinished non-terminal. If both derivations follow the same tree, they produce the same structure in a different order.

Consider the sentence:

The student checks the timetable.

A simplified grammar for this example includes:

Q → S

S → NP VP

NP → Det N

VP → Vtrans NP

Det → The | the

N → student | timetable

Vtrans → checks

Leftmost derivation

Q

⇒ S

⇒ NP VP

⇒ Det N VP

⇒ The N VP

⇒ The student VP

⇒ The student Vtrans NP

⇒ The student checks Det N

⇒ The student checks the timetable

Rightmost derivation

Q

⇒ S

⇒ NP VP

⇒ NP Vtrans NP

⇒ NP Vtrans Det N

⇒ NP Vtrans Det timetable

⇒ NP Vtrans the timetable

⇒ Det N checks the timetable

⇒ The student checks the timetable

Bracketed tree

(S

(NP (Det The) (N student))

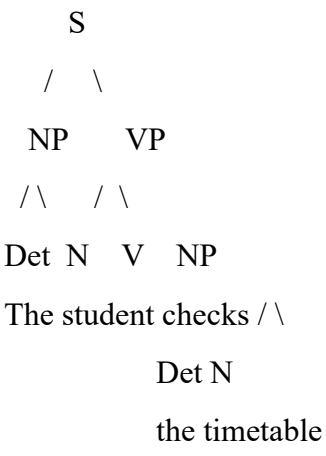
(VP

(V checks)

(NP (Det the) (N timetable))))

The tree has a sentence node, phrase nodes, preterminal nodes and terminal word nodes. If all of these are counted, this example contains 14 nodes. Its height is four edges from the sentence node to a terminal word. The derivation shown uses eight rule applications.

A tree diagram can be represented as:



2.2 Example queries and their structures

Query	Construction	Simplified bracketed structure
The exams begin tomorrow.	Declarative	S (NP The exams) (VP begin (AdvP tomorrow)))
Submit the form.	Imperative	IMP Submit (NP the form))
Does the portal show results?	Yes/no question	YNQ Does (NP the portal) (VPbase show results))
When will the exam begin?	Wh-adverbial question	WHQ When will (NP the exam) (VPbase begin))

Which room did you book?	Wh-object question	WHQ Which room did (NP you) (VPbase book __))

In the last example, the blank marks an understood object position associated with “Which room.” A feature-based grammar can record this relationship so the parser checks that the fronted phrase matches the missing object.

2.3 Structural ambiguity

The query **“Book the room near the library for the seminar”** has more than one possible structure.

In the first reading, “near the library” modifies “room”:

(VP

Book

(NP the

(Nbar room

(PP near the library)))

(PP for the seminar))

In another reading, “near the library” describes where the booking should occur:

(VP

(VP Book (NP the room))

(PP near the library)

(PP for the seminar))

The query **“Show the students with the late fee notice”** is also ambiguous. The PP “with the late fee notice” can describe which students should be shown, or it can describe how the showing should be carried out.

These ambiguities arise because PPs can attach to NPs or VPs. A parser can recognize that a sentence has multiple trees, but syntax alone may not identify the intended interpretation. Context and semantic information can help.

A basic membership test asks whether the grammar can derive the complete input from the start symbol. A chart parser can do this while recording partial analyses. The test is limited: it can report that a string is grammatical according to the grammar, but it cannot prove that the grammar covers all English or identify the speaker’s intended meaning.

Task 3 — Sentence-Level Constructions

A helpdesk must handle statements, commands and questions. These construction types differ in word order and in whether a subject is present.

Construction	Example	Main representation
Declarative	The library opens at nine.	S → NP VP
Imperative	Please check the timetable.	IMP → Please Vbase NP

Yes/no question	Can I book a room?	YNQ → Modal NP VPbase
Wh-subject question	Who submitted the form?	WHQ → WhNP VP
Wh-object question	Which room did you book?	WHQ → WhNP Aux NP VPbase
Wh-adverbial question	When will exams begin?	WHQ → WhAdv Aux NP VPbase
Relative clause	he exams that were postponed	NP → NP RelClause
Embedded question	Tell me whether the fee is due.	VP → Vwh NP WHQ

Auxiliary inversion places the auxiliary before the subject in many questions. For example, the declarative "You can book a room" becomes "Can you book a room?" by moving "can" before "you."

Wh-movement places a question word at the beginning. In "Which room did you book?" the object phrase appears before the auxiliary, while its understood position follows "book." A CFG can use a special rule for this pattern. Feature structures can record a gap and carry it through an embedded clause.

Relative clauses modify noun phrases. In "the exams that were postponed," "that were postponed" is a relative clause attached to "the exams." Coordination joins similar units: "fees and hostel charges" joins NPs, while "Fees are due and the library is open" joins clauses.

Adding each construction increases grammar size and can increase ambiguity. Relative clauses can attach at different points, and coordinated expressions can have different scopes. Rules should therefore be tested against examples of both grammatical and ungrammatical input.

Task 4 — Agreement

Agreement ensures that related words have compatible grammatical properties. This helpdesk grammar considers:

- **Subject–verb agreement:** "The student submits" versus "The students submit."
- **Determiner–noun agreement:** "This student" versus "These students."
- **Pronoun case:** "I booked a room" versus "Me booked a room."

A plain CFG can enforce agreement by multiplying categories. For example:

S → NPsg VPsg

S → NPpl VPpl

NPsg → DetSg Nsg

NPpl → DetPl Npl

NPsubj → PronSubj

NPobj → PronObj

This works for a small grammar, but the number of categories grows as more properties are represented. If number has two values, person has three, case has two and gender has two, a

category could have 2 × 3 × 2 × 2 = 24 versions. If ten rule templates were copied for each version, the inventory could require up to 240 productions before valence and sentence constructions are added.

Test sentence	Basic CFG	Agreement-aware grammar	Explanation
The student submits the form.	Accept	Accept	Singular subject and verb
The students submit the forms.	Accept	Accept	Plural subject and verb
I check my timetable.	Accept	Accept	First-person subject
They are in the library.	Accept	Accept	Plural subject and auxiliary
The students submits the form.	May accept	Reject	Subject and verb disagree
These student is late.	May accept	Reject	Determiner, noun and verb mismatch
Me booked a room.	May accept	Reject	Object-case pronoun used as subject
Them are waiting.	May accept	Reject	Object-case pronoun used as subject
Each students submit a form.	May accept	Reject	"Each" selects a singular noun
The form are ready.	May accept	Reject	Singular noun with plural verb

Feature-based rules provide a more compact alternative. A sentence rule can require the subject's agreement features to match those of the verb phrase. This lets the parser reject incompatible strings without enumerating every combination.

Task 5 — Subcategorization

Subcategorization, also called **valence**, describes the complements that a verb requires or permits.

Verb frame	Example	Grammar pattern
Intransitive: crash	The portal crashed.	VP → Vintr
Transitive: submit	Submit the form.	VP → Vtrans NP
Ditransitive: send	Send me the receipt.	VP → Vditrans NP NP
Sentential complement: tell	Tell me that the fee is paid.	VP → Vsent NP S
Vh-complement: tell	Tell me when the exam begins.	VP → Vwh NP WHQ
PP complement: apply	Apply for a hostel room.	VP → Vpp PP
Copular: be	The room is available.	VP → Vcop AP

A grammar with separate verb categories, such as **Vtrans** and **Vintr**, can reject invalid combinations. "The portal crashed the form" is rejected because "crash" is intransitive in the

intended meaning. “Submit” without an object may be rejected in a context where the transitive frame is required.

A **complement** is selected by the verb and helps complete its meaning. An **adjunct** adds optional information. In “book a room for Friday,” “a room” is the object complement of “book.” “For Friday” is an optional time phrase and can be treated as an adjunct. It can be omitted: “Book a room.” It can also move: “For Friday, book a room.”

Some verbs permit more than one frame. “Book” can take an NP alone (“Book a room”) or an NP plus a PP (“Book a room for Friday”). Listing each frame can increase grammar size and introduce ambiguity. Feature structures provide a compact way to record a verb’s subcategorization requirements.

Task 6 — Top-Down Parsing

A top-down parser starts from the grammar’s start symbol and predicts which rules could generate the input. A recursive-descent parser expands one non-terminal at a time, matches terminals against the sentence and backtracks when a choice fails.

```
parse(X, i):
  if X is a terminal:
    match token[i]
    return i + 1 if it matches; otherwise fail

  for each rule X → Y1 ... Ym:
    position = i
    try to parse Y1, then Y2, and so on
    if all symbols succeed:
      return the tree and final position
    otherwise backtrack and try another rule
```

For **“The student checks the timetable,”** the parser begins with Q, expands Q to S and S to NP VP. It parses “The student” as a noun phrase, then parses “checks the timetable” as a verb phrase. If the rules are ordered suitably, it can find the analysis without backtracking.

For **“Book the room near the library,”** the parser may first attach “near the library” to “room.” If it must find all parses, it must also try attaching the PP to the VP. With k PPs and two possible attachment sites for each, as many as 2^k combinations may need to be explored by a naive parser.

Left recursion can prevent ordinary recursive descent from making progress. A rule such as NP → NP PP calls NP again before consuming any input, causing a loop. One rewrite is:

NP → Nbar NPtail

NPtail → PP NPtail

NPtail → ε

A left-corner parser can also reduce unproductive predictions by requiring evidence from the input

before predicting a larger constituent. Memoization can prevent repeated analysis of the same substring.

Task 7 — Earley Parsing

The Earley algorithm is a chart parser. An Earley state has the form:

[A → α • β, i]

Here, A → αβ is a grammar rule, the dot marks the point reached in that rule and i records where the constituent began. The chart at position *j* contains states found after processing *j* words.

Earley parsing uses three main operations:

1. **Predictor:** When a non-terminal appears after the dot, add rules that expand it.
2. **Scanner:** When a terminal appears after the dot, compare it with the next input word and advance the dot if they match.
3. **Completer:** When a constituent finishes, advance states that were waiting for that constituent.

Consider **"The student checks the timetable."** Use this small grammar:

Q → S

S → NP VP

NP → Det N

VP → Vtrans NP

Det → the

N → student

N → timetable

Vtrans → checks

The chart indices run from 0 to 5.

S[0]

[Q → • S, 0] seed

[S → • NP VP, 0] predictor

[NP → • Det N, 0] predictor

[Det → • the, 0] predictor

S[1] after "The"

[Det → the •, 0] scanner

[NP → Det • N, 0] completer

[N → • student, 1] predictor

[N → • timetable, 1] predictor

S[2] after “student”

[N → student •, 1] scanner

[NP → Det N •, 0] completer

[S → NP • VP, 0] completer

[VP → • Vtrans NP, 2] predictor

[Vtrans → • checks, 2] predictor

S[3] after “checks”

[Vtrans → checks •, 2] scanner

[VP → Vtrans • NP, 2] completer

[NP → • Det N, 3] predictor

[Det → • the, 3] predictor

S[4] after “the”

[Det → the •, 3] scanner

[NP → Det • N, 3] completer

[N → • student, 4] predictor

[N → • timetable, 4] predictor

S[5] after “timetable”

[N → timetable •, 4] scanner

[NP → Det N •, 3] completer

[VP → Vtrans NP •, 2] completer

[S → NP VP •, 0] completer

[Q → S •, 0] completer; accepted

A real parser stores back-pointers that show which rules and child states created each completed state. When a phrase has several analyses, the parser can store them as alternatives in a packed parse forest rather than duplicating every structure.

Earley parsing handles left-recursive rules because it records states in the chart and avoids repeatedly adding the same state at the same position. For a general CFG, its worst-case time is $O(n^3)$ and its chart space is $O(n^2)$, where *n* is the sentence length. These are theoretical bounds, not measured run times for this helpdesk grammar.

Task 8 — Feature Structures and Unification

A feature structure records grammatical information as attribute–value pairs. A **feature path** reads a value inside a structure; for example, AGR.NUM refers to the number value inside the agreement feature. **Reentrancy** means that two parts of a structure share the same information.

For example, the subject and verb phrase can share an agreement structure.

Sample lexical feature structures:

student: [CAT N, NUM sg, PERSON 3]

students: [CAT N, NUM pl, PERSON 3]

this: [CAT Det, NUM sg]

these: [CAT Det, NUM pl]

checks: [CAT V, AGR [NUM sg, PERSON 3], SUBCAT <NP>]

check: [CAT V, AGR [NUM pl, PERSON 3], SUBCAT <NP>]

me: [CAT Pron, CASE obj, NUM sg, PERSON 1]

I: [CAT Pron, CASE subj, NUM sg, PERSON 1]

Unification combines feature structures if they contain no conflicting values.

- [NUM=sg] unifies with [NUM=sg], producing [NUM=sg].
- [PERSON=3, NUM=pl] unifies with the same bundle and keeps those values.
- [NUM=pl] fails to unify with [NUM=sg] because the number values conflict.

A compact feature grammar can express agreement and valence:

S[AGR=?a] → NP[CASE subj, AGR ?a] VP[AGR ?a]

NP[AGR ?a] → Det[NUM ?n] N[NUM ?n, AGR ?a]

VP[AGR ?a] → V[AGR ?a, SUBCAT <NP>] NP

VP[AGR ?a] → V[AGR ?a, SUBCAT <PP>] PP

VP[AGR ?a] → V[AGR ?a, SUBCAT <NP, NP>] NP NP

VP[AGR ?a] → V[AGR ?a, SUBCAT <S>] S

In “The students check the timetable,” the subject NP has plural agreement features. The verb “check” unifies with those features, and the object NP satisfies the verb’s subcategorization requirement. In “The students checks the timetable,” the plural subject conflicts with the singular verb form, so unification fails. A subject pronoun must also have CASE=subj, rejecting “Me booked a room.”

Features replace many duplicated categories, although they add some processing cost. In an Earley parser, feature unification can occur during completion. Feature values can create additional chart states, so a parser may use structure sharing and packed alternatives to reduce duplication.

Task 9 — Probabilistic Context-Free Grammars

A probabilistic CFG assigns a probability to each rule. For every left-hand-side category, the probabilities of its alternative rules must sum to one. The probability of a parse tree is the product of the probabilities of its rules. The probability of a sentence is the sum of the probabilities of all trees that produce it.

9.1 Example rule estimates

Suppose a small annotated treebank contains 20 relevant VP expansions and 20 NP expansions. The maximum-likelihood estimate for a rule is:

P(rule) = count(rule) / count(all rules with the same left-hand side)

Additive smoothing can avoid zero probabilities. With smoothing value α = 0.5 and K alternatives for a category:

P(rule) = (count(rule) + α) / (total count + αK)

Production	**Count**	**Total for category**	**Smoothed probability**
VP → V NP	12	20	12.5 / 21 = 0.595
VP → VP PP	8	20	8.5 / 21 = 0.405
NP → Det N	16	20	16.5 / 22 = 0.750
NP → NP PP	4	20	4.5 / 22 = 0.205
NP → Pron	0	20	0.5 / 22 = 0.023
NP → Proper	0	20	0.5 / 22 = 0.023

These counts illustrate how the calculation works. A production system should estimate rule probabilities from a representative annotated corpus. It should also reserve probability for unseen or rare words.

9.2 Probability of alternative PP attachments

Consider **“Book the room near the library.”** Assume the following illustrative probabilities.

For NP attachment:

P(VP → V NP) = 0.60

P(NP → Det Nbar) = 0.75

P(Nbar → N PP) = 0.25

P(PP → P NP) = 0.80

Tree probability = 0.60 × 0.75 × 0.25 × 0.80 = 0.090

For VP attachment:

P(VP → VP PP) = 0.40

P(VP → V NP) = 0.60

P(NP → Det N) = 0.75

P(PP → P NP) = 0.80

Tree probability = 0.40 × 0.60 × 0.75 × 0.80 = 0.144

If these are the only two parses and their word probabilities are equal, the sentence probability is:

P(sentence) = 0.090 + 0.144 = 0.234

The VP attachment has the larger probability in this example. Different corpus counts could produce a different ranking.

9.3 Viterbi CKY algorithm

The Viterbi version of CKY finds the highest-probability parse after the grammar is converted to Chomsky Normal Form (CNF).

Initialize each diagonal chart cell with lexical rule probabilities.

For spans of length 2 through n:

  For each span [i, j]:

    For every split point k:

      For each binary rule A → B C:

        candidate =

          P(A → B C) × V[i, k, B] × V[k, j, C]

        Keep the highest candidate and its back-pointer.

Return the best S entry for the complete sentence.

Follow back-pointers to recover the best tree.

A longer rule such as VP → V NP PP can be converted into binary rules by introducing an intermediate symbol:

VP → V X

X → NP PP

The extra rule can be assigned probability 1 so the original derivation probability is preserved.

PCFGs make simplifying assumptions. A rule probability is context-free and position-invariant: the probability of VP → VP PP does not directly depend on the verb, discourse or distance. Multiplying several probabilities below one can favor shorter trees. A basic PCFG also lacks lexical sensitivity, so it may choose a common structural pattern even when the nearby words suggest another attachment. Lexicalized models can help, but they require more data.

Task 10 — Implement and Test the Parsers

A complete implementation should compare:

1. A recursive-descent parser.
2. An Earley chart parser.

3. A feature-aware parser for agreement and subcategorization.
4. A Viterbi parser using PCFG rule probabilities.

The following 30 sentences form a test set. They cover statements, commands, questions, coordination, relative clauses, ambiguous attachments and agreement errors.

1. The student checks the timetable.
2. The students check the timetable.
3. I need the exam schedule.
4. The portal crashed yesterday.
5. Submit the application form.
6. Send me the receipt.
7. Tell me that the fee is paid.
8. Apply for a hostel room.
9. The library is open today.
10. Does the portal show results?
11. Can I book a room?
12. When will the exams begin?
13. Which room did you reserve?
14. Who submitted the form?
15. Please check my account.
16. Fees and hostel charges are due.
17. The exams that were postponed will start Monday.
18. Book the room near the library.
19. Show the students with the late fee notice.
20. Book the room near the library for the seminar.
21. The students submits the form.
22. These student is late.
23. Me booked a room.
24. The portal crashed the form.
25. Submit.
26. Tell me when the exam begins.
27. Can you tell me whether the library is open?
28. The staff member checks the placement portal.
29. Reserve the room for the seminar after lunch.
30. Find the result in the student account.

For each parser, record whether the sentence was accepted, the number of parses, the number of

chart states or recursive calls, and run time. Run each test repeatedly in the same environment and report median time rather than relying on one measurement. Include a sentence of up to 20 words and cases with repeated PPs to test growth.

The results should be stored in a table with columns such as sentence_id, length, parser, accepted, parse_count, states and median_ms. Plot sentence length against state count or run time. These measurements must come from actually running the implementation; theoretical complexity should not be reported as observed timing data.

Task 11 — Compare, Recommend and Defend the Parsing Design

A top-down parser is easy to trace and can work well with a small deterministic grammar. It is vulnerable to left recursion, backtracking and repeated work. Earley parsing is a stronger recognizer for a CFG with recursion and ambiguity because it records partial results in a chart. A PCFG can rank the parses found by the recognizer, but it does not replace the grammar or parser.

The recommended initial system is:

Feature-based CFG

↓

Earley chart parser

↓

PCFG ranking of valid parses

The feature grammar checks agreement, case and verb complements. Earley parsing provides chart-based recognition and can preserve ambiguity. PCFG probabilities select a likely parse from the alternatives.

The design is interpretable and can be extended as helpdesk coverage grows. Its limitations are incomplete hand-written coverage, sparse probability estimates, and difficulty handling misspellings and fragments. As the helpdesk receives more varied requests, collecting and annotating representative examples becomes more valuable than adding increasingly specific rules. A hybrid system can retain explicit linguistic constraints while learning parse preferences from data.

Task 12 — Day Four Design Challenge

12.1 Impact analysis

The new example is:

“Hi, can you tell me when the exams that were postponed will be held, and also whether the hostel fee is due?”

It contains several constructions that may not be covered by the original grammar:

- A greeting fragment: “Hi”
- A modal question: “Can you tell me…”
- An embedded wh-question: “when the exams … will be held”

- A relative clause: “that were postponed”
- A passive auxiliary chain: “will be held”
- Coordination of embedded clauses
- Informal punctuation or missing punctuation

The original grammar assumes relatively complete, grammatical sentences and a controlled vocabulary. Real messages may be incomplete, misspelled or conversational. A strict parser may reject the entire message if it cannot find a complete root parse.

Longer input creates more possible spans and attachments. Earley parsing has a theoretical $O(n^2)$ chart-space bound and $O(n^3)$ worst-case time for a fixed CFG. The number of full parse trees can still grow rapidly under ambiguity, even if shared structures are stored compactly.

12.2 Evaluate the original design

New message	**Likely first difficulty**	**Reason**
Hi, can you tell me when the exams that were postponed will be held?	Greeting and outer question	The grammar may not accept a greeting fragment or nested questions
paid the fee but still cannot see my result.	Auxiliary and negation	The grammar may lack negation and auxiliary patterns
Hostel fee due tomorrow?	Fragment root	The message has no overt finite verb

Earley parsing can share work across analyses, but it cannot parse a construction that is absent from its grammar. A strict recognizer may fail when no full sentence spans all input words. The parser should therefore support a recovery mode that identifies meaningful partial spans.

12.3 Adaptation choice: modify the design

The system should be modified rather than replaced. Retain the feature grammar and Earley chart parser, and add:

1. Rules for greetings and short fragments.
2. Rules for auxiliary chains, negation and passive voice.
3. Rules for embedded wh- and whether-clauses.
4. Feature constraints for relative-clause gaps.
5. Coordination at clause and verb-phrase levels.
6. Tokenization that tolerates missing punctuation and an unknown-word category.
7. Partial parsing that returns recognized spans and marks them as incomplete or uncertain.

The ranking model should be trained on a larger set of reviewed helpdesk examples. The training data should include complete sentences, fragments and representative conversational messages. A held-out evaluation set should measure full parse coverage, partial coverage, best-parse accuracy, ambiguity and median latency.

12.4 Evidence-based defence

The CFG captures common query structures and recursive modification. Feature structures reject agreement and valence errors without copying every rule. The Earley chart trace demonstrates

how a chart parser handles recursive rules by reusing partial results. The PCFG example demonstrates how rule probabilities can rank two legal PP attachments.

The new messages expose gaps in the original coverage, so the design needs additional constructions and partial parsing. A larger hand-written grammar alone would be difficult to maintain. Extending the existing architecture and adding annotated examples supports broader coverage while keeping the system interpretable.

Remaining limitations include semantic ambiguity, new vocabulary, code-switching and misspellings. A syntactically likely parse may still fail to represent the student's intention. If two plausible parses would lead to different helpdesk actions, the assistant should ask a clarification question.

Conclusion

A context-free grammar provides a clear foundation for analyzing common English helpdesk queries. Agreement and subcategorization features improve the grammar by checking subject–verb compatibility, pronoun case and verb complements. Top-down parsing explains goal-directed search, but left recursion and ambiguity can cause repeated work. Earley parsing addresses these problems through chart-based processing, while PCFG probabilities rank competing parses.

For conversational, multi-clause input, the recommended system is a modified hybrid: a feature-based CFG, Earley chart parsing, probabilistic ranking and a partial-parse recovery layer. Its performance should be evaluated with a documented test set and actual parser measurements. The system's accuracy will depend on grammar coverage, representative annotated data and ongoing testing.

References

1. Jurafsky, D. and Martin, J. H. *Speech and Language Processing*, 3rd edition draft. Chapters on CFGs, parsing and PCFGs.
2. Earley, J. (1970). "An Efficient Context-Free Parsing Algorithm." *Communications of the ACM*, 13(2), 94–102.
3. Shieber, S. (1986). *An Introduction to Unification-Based Approaches to Grammar*. CSLI Publications.
4. Manning, C. D. and Schütze, H. (1999). *Foundations of Statistical Natural Language Processing*. MIT Press.

6:00 PM

MARK DISTRIBUTION

Task	**Marks**
Task 1 – Design a Context-Free Grammar for English Syntax	5
Task 2 – Context-Free Rules, Derivations and Parse Trees	3
Task 3 – Sentence-Level Constructions	4
Task 4 – Agreement	3
Task 5 – Subcategorization	4
Task 6 – Top-Down Parsing	4
Task 7 – Earley Parsing	5
Task 8 – Feature Structures and Unification	4
Task 9 – Probabilistic Context-Free Grammars	5
Task 10 – Implement and Test the Parsers	3
Task 11 – Compare, Recommend and Defend the Parsing Design	2
Subtotal	**42**
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**